الدوال السبع وتحليل التعقيد الزمني (Big-O)

مقدمة سريعة

تحليل الخوارزميات يهتم بالسؤال: **إذا حجم المدخلات n كبر، شكد يزيد وقت التنفيذ؟**

نعبر عن هذا باستخدام **Big-O Notation** مثل $O(1), O(\log n), O(n)$ ... إلخ.
كل نوع من هذه التعقيدات يمثل **شكل معيّن لنمو الزمن** مع زيادة n.

في هذه الملزمة نغطي ٧ **دوال أساسية** تستخدم في أغلب تحليلات الخوارزميات:

1. الدالة الثابتة
2. الدالة اللوغاريتمية
3. الدالة الخطية
4. دالة $n \log n$
5. الدالة التربيعية
6. الدالة التكعيبية وكثيرات الحدود
7. الدالة الأسية

مع أمثلة من الحياة اليومية ومن كود C++.

1. الدالة الثابتة $O(1)$ – Constant Time

الفكرة الأساسية

- الزمن **لا يعتمد** على حجم المدخل n.
- مهما كبر حجم البيانات، عدد الخطوات يبقى تقريباً ثابت.

مثال حياتي

- زر الكهرباء يشغّل 1 مصباح أو 1000 مصباح بنفس ضغطة الزر.

مثال برمجي (C++)

```cpp
int getFirst(int arr[], int n) {
    return arr[0];  // وصول مباشر لأول عنصر
}
// التعقيد الزمني: O(1)
```

أمثلة نموذجية

- الوصول لعنصر في مصفوفة بالفهرس `arr[i]`.
- `push`, `pop`, `top` في Stack (تنفيذ صحيح).
- `enqueue`, `dequeue` في Queue (تنفيذ صحيح).

> هدفنا في هياكل البيانات: نجعل أكبر عدد ممكن من العمليات O(1).

---

2. الدالة اللوغاريتمية $O(\log n)$ – Logarithmic Time

الفكرة الأساسية

- في كل خطوة **نقصّ المشكلة إلى النصف** (أو نسبة ثابتة).
- عدد الخطوات تقريباً يساوي $\log_2 n$.

مثال حياتي

- تبحث عن كلمة في قاموس ورقي:
  - تفتح وسط القاموس → إذا الكلمة في النصف الثاني، ترمي النصف الأول وتكمل على الثاني.
  - كل خطوة تخلص من نصف الصفحات.

مثال برمجي – Binary Search

```cpp
int binarySearch(int arr[], int n, int target) {
    int left = 0, right = n - 1;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target)  left = mid + 1;
        else                    right = mid - 1;
    }
    return -1;
}
// التعقيد الزمني: O(log n)
// شرط مهم: المصفوفة مرتبة مسبقاً
```

ملاحظات مهمة

- تغيير الأساس (2 أو 10) لا يهم في Big-O، لأن الفرق ثابت.
- لو $n = 1,000,000,000$، فالـ $\log_2 n$ تقريباً 20 فقط → عدد خطوات قليل جداً.

3. الدالة الخطية $O(n)$ – Linear Time

الفكرة الأساسية

- الزمن **يتناسب طردياً** مع عدد العناصر.
- عملية واحدة تقريباً لكل عنصر.

مثال حياتي

- قراءة كتاب صفحة صفحة.
  - 10 صفحات → 10 دقائق.
  - 100 صفحة → 100 دقيقة.

مثال برمجي – جمع عناصر مصفوفة

```cpp
int sumArray(int arr[], int n) {
    int sum = 0;
    for (int i = 0; i < n; i++)
        sum += arr[i];
    return sum;
}
// التعقيد الزمني: O(n)
```

مثال برمجي – البحث الخطي

```cpp
int linearSearch(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target)
            return i;
    }
    return -1;
}
// أسوأ حالة: نفحص كل العناصر → O(n)
```

ملاحظة

- **أي خوارزمية لازم تمر على كل العناصر مرة واحدة على الأقل** لا يمكن أن تكون أسرع من O(n).

4. دالة $O(n \log n)$ – N-Log-N Time

الفكرة الأساسية

- مزيج بين خطي ولوغاريتمي:
  - عندنا $\log n$ مستويات (تقسيم متكرر).
  - في كل مستوى نعالج تقريباً n عنصر.
- يظهر كثيراً في **خوارزميات الفرز الجيدة**.

مثال حياتي

- ترتيب أوراق اللعب (شدة) بطريقة ذكية:
  - تقسّم الأوراق إلى مجموعات صغيرة.
  - ترتب كل مجموعة.
  - تدمج المجموعات بشكل مرتب.

مثال برمجي – الفكرة العامة لـ Merge Sort

```cpp
void mergeSort(int arr[], int left, int right) {
    if (left >= right) return;          // حالة التوقف
    int mid = (left + right) / 2;
    mergeSort(arr, left, mid);          // نصف أول
    mergeSort(arr, mid + 1, right);     // نصف ثاني
    merge(arr, left, mid, right);       // دمج مصفوفتين مرتبتين
}
// التعقيد الزمني الإجمالي: O(n log n)
```

لماذا مهم؟

- فرق كبير بين $O(n^2)$ و $O(n \log n)$ عندما يكون n كبيراً.
- مثال: لو $n = 10^5$:
  - $n^2 = 10^{10}$ (بطيء جداً)
  - $n \log n \approx 10^5 \times 17 = 1.7 \times 10^6$ (مقبول)

5. الدالة التربيعية $O(n^2)$ – Quadratic Time

الفكرة الأساسية

- عادة تأتي من **حلقتين متداخلتين**.
- لكل عنصر نمر على كل العناصر.

مثال حياتي

- حفلة تعارف فيها n أشخاص، وكل شخص يسلم على كل الموجودين:
  - العدد الكلي للمصافحات تقريباً $n(n-1)/2$ → تربيعي.

مثال برمجي – **Nested Loop** بسيط

```cpp
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        // عملية بسيطة
    }
}
// التعقيد الزمني: O(n^2)
```

مثال برمجي – **Bubble Sort**

```cpp
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j+1])
                swap(arr[j], arr[j+1]);
        }
    }
}
// التعقيد الزمني في أسوأ حالة: O(n^2)
```

ملاحظات

- مجموع $1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2} = O(n^2)$.
- مقبول فقط للمجموعات الصغيرة؛ للمدخلات الكبيرة يصبح بطيئاً جداً.

6. الدالة التكعيبية وكثيرات الحدود $O(n^3), O(n^d)$

الفكرة الأساسية

- تأتي من 3 **حلقات متداخلة** أو أكثر.
- كثيرات الحدود بشكل عام:

$$^{d}a_{d}n + \cdots + {}^{2}a_{2}n + a_{1}n + {}_{0}a = f(n)$$

- في Big-O نهتم بأكبر درجة فقط: $O(n^d)$.

مثال حياتي

- اختيار طقم ملابس:
  - n قمصان، n بناطيل، n أحذية.
  - نجرب كل الاحتمالات: $^{3}n = n \times n \times n$.

مثال برمجي – ثلاث حلقات متداخلة

```cpp
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        for (int k = 0; k < n; k++) {
            // عملية معينة
        }
    }
}
// التعقيد الزمني: O(n^3)
```

ملاحظات

- كلما زادت درجة كثير الحدود، زاد الزمن بسرعة.
- نفضّل دائماً الدرجات الأقل (خاصة ≤ 2) إذا أمكن.

7. الدالة الأسية $O(2^n)$ – Exponential Time

الفكرة الأساسية

- الزمن **يتضاعف تقريباً** مع كل زيادة صغيرة في n.
- خطيرة جداً وتنفجر بسرعة.

مثال حياتي

- انتشار إشاعة:
  - شخص يخبر 2.
  - كل واحد منهم يخبر 2.
  - بعد k خطوات، عدد الناس تقريباً 2^k.

مثال برمجي – Fibonacci (الطريقة الساذجة)

```cpp
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);  // استدعاءان في كل مرة
}
// 2 ≈ عدد الاستدعاءات ^n → تعقيد أسي
```

ملاحظات

- خوارزميات O(2^n) تقريباً **غير عملية** إلا إذا كان n صغير جداً.
- نحاول دائماً إيجاد خوارزمية بديلة (Dynamic Programming, Memoization) لتقليل التعقيد.

مقارنة معدلات النمو (من الأفضل إلى الأسوأ)

الترتيب	التعقيد	الاسم
1	$O(1)$	Constant – ثابتة
2	$O(\log n)$	Logarithmic – لوغاريتمية
3	$O(n)$	Linear – خطية
4	$O(n \log n)$	N-Log-N – خطية-لوغاريتمية
5	$O(n^2)$	Quadratic – تربيعية
6	$O(n^3)$ أو $O(n^d)$	تكعيبية / كثيرات حدود
7	$O(2^n)$	Exponential – أسية

تفسير تشبيهي بسيط

- **المنطقة الخضراء:**
  - $O(1)$, $O(\log n)$, $O(n)$ → ممتازة وعملية.
- **المنطقة الصفراء:**
  - $O(n \log n)$ → جيدة جداً وغالباً أفضل ما نستطيع في الفرز.
- **المنطقة الحمراء:**
  - $O(n^2)$, $O(n^3)$ → بطيئة للمدخلات الكبيرة.
  - $O(2^n)$ → كارثية إلا لـ n صغير جداً.

تحليل الخوارزميات (Analysis of Algorithms)

هذا الموضوع يجاوب على السؤال المهم: **"كيف نحكم على الخوارزمية: هل هي ممتازة أم سيئة؟"**

الهدف: نتعلم **طرق علمية** لتقييم الخوارزميات بشكل مستقل عن:

- نوع الجهاز (لابتوب قديم أو سوبر كمبيوتر)
- لغة البرمجة (C++ أو Python)
- نظام التشغيل (Windows أو Linux)

نركز على **معدل النمو** عند زيادة حجم المدخلات n.

1️⃣ الدراسات التجريبية (Experimental Studies)

الفكرة الأساسية

طريقة "ساعة الإيقاف" ⏱️: نشغّل الكود ونقيس الوقت الفعلي.

الخطوات

1. **برمج** الخوارزمية كاملة
2. **شغّلها** على مدخلات مختلفة (صغيرة ومتوسطة وكبيرة)
3. **سجّل** الوقت لكل حجم مدخل
4. **ارسم** رسم بياني (محور x = حجم المدخل n، محور y = الوقت t)

مثال تشبيهي 🚗

مثل سباق سيارات: نجيب سيارتين ونسابقهم على نفس الطريق ونشوف أيهما يوصل أسرع.

المشاكل الثلاث الكبيرة ❌

المشكلة	التوضيح
ظلم الأجهزة	كود سيء على جهاز خارق = يبدو سريع! كود عبقري على لابتوب قديم = يبدو بطيء!
لازم تبرمج كل شي	تضيع وقت طويل في البرمجة، وممكن الفكرة تطلع غلط بعد ما تخلص
مدخلات محدودة	تجرب 1000 حالة والكود تمام، بس الحالة 1001 تعلّق البرنامج

الخلاصة

الدراسات التجريبية **مفيدة** بس **غير عادلة** ولا تعطي صورة كاملة.

2️⃣ العمليات الأولية (Primitive Operations)

الفكرة الأساسية

بدل ما نحسب **الوقت** (يختلف من جهاز لجهاز)، نحسب **عدد الخطوات** (ثابت لكل الأجهزة).

تعريف العملية الأولية

تعليمة بسيطة جداً تأخذ **وقت تنفيذ ثابت** $O(1)$.

أمثلة على العمليات الأولية

```cpp
// ✅ كلها عمليات أولية
int x = 5;              // تعيين قيمة
int y = x + 10;         // عملية حسابية
if (x > y)              // مقارنة
cout << arr[3];         // فهرسة في مصفوفة
return x;               // رجوع من دالة
func(x);                // استدعاء دالة
```

ليست عمليات أولية ❌

```cpp
// ❌ هذي مو عمليات أولية
for (int i = 0; i < n; i++)  // حلقة (تعتمد على n)
sort(arr, arr+n);            // ترتيب (عمليات كثيرة)
```

مثال: حساب العمليات الأولية

```cpp
int sum = 0;            // 1 عملية (تعيين)
for (int i = 0; i < n; i++) {  // n مرة
    sum += arr[i];      // n × عمليتان (فهرسة + جمع)
}
return sum;             // 1 عملية (رجوع)
```

إجمالي العمليات: $2 + 3n = 1 + (2 + n(1 + 1$

في Big-O: نقول $O(n)$ (نتجاهل الثوابت)

3️⃣ تحليل الحالات: Best, Average, Worst

مثال عملي: البحث الخطي (Linear Search)

```cpp
} int linearSearch(int arr[], int n, int target)
} for (int i = 0; i < n; i++)
if (arr[i] == target)
وجدناه! // ;return i
{
مو موجود // ;return -1
{
```

الحالة	الوصف	التعقيد	مثال
Best Case (أفضل حالة)	العنصر موجود في أول موضع	O(1)	ندور على 5 والمصفوفة
Average Case (الحالة المتوسطة)	نفترض توزيع احتمالي	O(n/2)=O(n)	صعب حسابياً
Worst Case (أسوأ حالة)	العنصر آخر واحد أو مو موجود	O(n)	ندور على 9 والمصفوفة

لماذا نركز على Worst Case؟ 🎯

تشبيه المهندس والجسر 🌉

المهندس يصمم الجسر ليتحمل:

- ❌ متوسط وزن السيارات؟ لا!
- ✅ أثقل شاحنة ممكنة؟ نعم!

السبب: إذا الكود سريع في أسوأ حالة → مضمون يكون سريع في كل الحالات.

صعوبة Average Case

```text
Average = Σ (احتمال الحالة × وقتها)
```

هذا يحتاج:

- تحديد توزيع احتمالي (مين أكثر: مدخلات كبيرة أو صغيرة؟)
- رياضيات معقدة (تكامل، احتمالات)
- افتراضات قد تكون غير واقعية

لذلك Worst Case **أسهل وأضمن**.

4️⃣ الترميز المقارب (Asymptotic Notation)

الفكرة الأساسية: "الصورة الكبيرة" 🔍

لما يكبر n كتير (مليون، مليار...)، التفاصيل الصغيرة ما تفرق:

- $5n$ و $100n$ **نفس الشي** في Big-O ← $O(n)$
- $1000n + {}^2n$ ← الـ ${}^2n$ يسيطر ← $O(n^2)$

نهمل:

- الثوابت (5, 100, 1000...)
- الحدود الصغيرة ($1000n$ صغير مقابل ${}^2n$)

📐 عائلة Big الثلاثة

أ) Big-O (الحد الأعلى) - "السقف" 🔺

التعريف الرياضي:

$O(g(n)) = f(n)$ يعني يوجد ثوابت $0 < c$ و ${}_0n \geq 1$ بحيث:

$$ {}_0n \leq g(n), \quad \forall n \cdot c \geq f(n) $$

بالعربي: الكود **ما راح يصير أبطأ** من هذا الحد.

مثال:

```cpp
// هذا الكود O(n)
for (int i = 0; i < n; i++) {
    cout << i;
}
// لأن: f(n) = n ≤ 1·n
```

ب) Big-Omega (الحد الأدنى) - "الأرضية" 🔻

التعريف الرياضي:

$\Omega(g(n)) = f(n)$ يعني يوجد $c > 0$ و $n_0 \geq 1$ بحيث:

$$n_0 \leq g(n), \quad \forall n \cdot c \leq f(n)$$

بالعربي: الكود **على الأقل** يحتاج هذا الوقت.

مثال: البحث في مصفوفة غير مرتبة

- **Worst Case:** $O(n)$ (نفحص كل العناصر)
- **Best Case:** $\Omega(1)$ (نلقاه أول واحد)

ج) Big-Theta (الحد الدقيق) - "بالضبط" 🎯

التعريف:

$\Theta(g(n)) = f(n)$ يعني $O(g(n)) = f(n)$ و $\Omega(g(n)) = f(n)$

بالعربي: الكود ينمو **بهذا المعدل بالضبط**، لا أسرع ولا أبطأ.

مثال:

```cpp
// هذا Θ(n²)
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        cout << i + j;
    }
}
// Best = Worst = n²
```

ج) Big-Theta (الحد الدقيق) - "بالضبط" 🎯

التعريف:

$\Theta(g(n)) = f(n)$ يعني $O(g(n)) = f(n)$ و $\Omega(g(n)) = f(n)$

بالعربي: الكود ينمو **بهذا المعدل بالضبط**، لا أسرع ولا أبطأ.

مثال:

```cpp
// هذا Θ(n²)
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        cout << i + j;
    }
}
// Best = Worst = n²
```

📊 جدول المقارنة: Big-O vs Omega vs Theta

الترميز	المعنى	التشبيه	مثال
$O(g(n))$	الحد الأعلى	"بالكثر" / Upper Bound	سرعتي بالسيارة ≤ 120 كم/س
$\Omega(g(n))$	الحد الأدنى	"عالأقل" / Lower Bound	عمري ≥ 18 سنة
$\Theta(g(n))$	الحد الدقيق	"بالضبط" / Tight Bound	راتبي = $1000 بالضبط

🎹 أمثلة عملية على التحليل

مثال **1:** دالة بسيطة

```cpp
int findMax(int arr[], int n) {
    int max = arr[0];          // 1 عملية
    for (int i = 1; i < n; i++) {  // n-1 مرة
        if (arr[i] > max)      // 1 مقارنة
            max = arr[i];      // 1 تعيين (في أسوأ حالة)
    }
    return max;                // 1 عملية
}
```

التحليل:

- العمليات الكلية: $2n = 1 + 2 \times (1 - n) + 1$
- **Big-O:** $O(n)$
- **Big-Theta:** $\Theta(n)$

مثال **2:** حلقات متداخلة

```cpp
void printPairs(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << arr[i] << " " << arr[j];
        }
    }
}
```

التحليل:

- الحلقة الخارجية: n مرة
- الحلقة الداخلية: n مرة لكل تكرار
- **المجموع:** $n \times n = n^2$
- **Big-Theta:** $\Theta(n^2)$

مثال **3:** حلقة متناقصة

```cpp
void mystery(int n) {
    for (int i = n; i > 0; i = i / 2) {
        cout << i << " ";
    }
}
```

التحليل:

- i يتناقص: $n, n/2, n/4, ..., 1$
- عدد الخطوات = $\log_2(n)$
- **Big-O:** $O(\log n)$

📏 قواعد تبسيط Big-O

القاعدة 1: نتجاهل الثوابت

```text
5n = O(n)
100n = O(n)
n/2 = O(n)
```

القاعدة 2: نأخذ الحد الأكبر فقط

```text
f(n) = 3n² + 100n + 50
→ نأخذ n² فقط
→ O(n²)
```

القاعدة 3: في حلقات متداخلة نضرب

```cpp
for (i...)        // O(n)
  for (j...)      // O(n)
    عملية ثابتة   // O(1)
→ O(n × n × 1) = O(n²)
```

القاعدة 4: في حلقات متتابعة نجمع

```cpp
for (i...)  // O(n)
for (j...)  // O(n)
→ O(n + n) = O(2n) = O(n)
```

🔬 الفرق بين Polynomial و Exponential

Polynomial Time (كثيرات الحدود)

- $O(n), O(n^2), O(n^3), \ldots$
- **قابلة للحل** (Tractable)
- تعتبر "فعّالة" نسبياً

Exponential Time (الأسية)

- $O(2^n), O(3^n), O(n!)$
- **غير قابلة للحل عملياً** (Intractable)
- حتى لو $n = 50$ يأخذ ملايين السنين!

📈 ترتيب الدوال حسب معدل النمو

من **الأسرع** (الأفضل) إلى **الأبطأ** (الأسوأ):

$$O(n!) > O(2^n) > O(n^3) > O(n^2) > O(n \log n) > O(n) > O(\log n) > O(1)$$

جدول المقارنة عند n = 1000

التعقيد	العمليات	الوصف
$O(1)$	1	فوري ⚡
$O(\log n)$	~10	ممتاز 🟢
$O(n)$	1,000	جيد 🟢
$O(n \log n)$	~10,000	مقبول 🟡
$O(n^2)$	1,000,000	بطيء 🔴
$O(2^n)$	1000^2	كارثة 💀

🎯 أسئلة شائعة

س: ليش نتجاهل الثوابت؟

ج: لأن عند n كبيرة جداً، الفرق بين $5n$ و $100n$ ما يفرق مقابل الفرق بين n و n^2.

س: متى نستخدم Big-Omega؟

ج: لما نريد نثبت أن الخوارزمية **ما تكدر تصير أسرع** من حد معين.

س: هل O(n) دائماً أسرع من O(n²)؟

ج: نعم، **عند n كبيرة**. بس لو n صغير جداً، ممكن خوارزمية $O(n^2)$ تكون أسرع بسبب الثوابت.